

Experiment 1

Data Preprocessing and Feature Engineering

Aim:

To perform data preprocessing and feature engineering by handling missing values, normalizing numerical features, encoding categorical variables, and visualizing the data, in order to improve data quality and prepare it for effective machine learning model development.

Theory:

In machine learning, raw data obtained from real-world sources is rarely suitable for direct use in algorithms. Such data often contains missing values, inconsistent feature scales, noise, and categorical variables that models cannot process directly. Data preprocessing is therefore a crucial step that involves cleaning, transforming, and organizing the dataset so that it becomes consistent and suitable for learning. Along with preprocessing, feature engineering helps in improving model performance by creating meaningful features that better represent underlying patterns in the data.

Handling Missing Data

Missing data occurs when values are absent for certain attributes in a dataset and is a common challenge in real-world data analysis. Missing values may arise due to data collection errors, incomplete observations, system failures, or random occurrences. If left untreated, missing data can distort statistical analysis and negatively affect the performance of machine learning models. To address this issue, missing values are first identified and analyzed. Numerical attributes are commonly handled using statistical imputation techniques such as mean or median substitution, while categorical attributes are often filled using the most frequent category. In cases where an attribute contains an excessive number of missing values, it may be removed to maintain data reliability. Proper handling of missing data ensures dataset completeness and consistency before further processing.

Normalization

Normalization is a preprocessing technique used to rescale numerical features so that they lie within a comparable range. In many datasets, numerical attributes may have significantly different scales, which can cause learning algorithms to be biased toward features with larger magnitudes. Normalization addresses this issue by transforming feature values using scaling techniques such as min-max normalization or standardization. This process improves numerical stability, speeds up model convergence, and ensures that all numerical features contribute equally to the learning process.

Min-max normalization, (usually called feature scaling) performs a linear transformation on the original data. This technique gets all the scaled data in the range $[0,1]$. The formula to achieve this is the following:

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Min-max normalization preserves the relationships among the original data values. The cost of having this bounded range is that we will end up with smaller standard deviations, which can suppress the effect of outliers.

Categorical Encoding

Many real-world datasets contain categorical variables represented as text or labels, whereas most machine learning algorithms require numerical input. Categorical encoding is the process of converting categorical data into numerical representations without losing meaningful information. Common encoding techniques include label encoding and one-hot encoding. Proper encoding allows categorical features to be effectively incorporated into machine learning models while preserving their interpretability and influence on predictions.

Traditional Encoding Techniques

- * One-Hot Encoding:

Converts each category into binary variables (0/1) indicating presence or absence. K-1 dummy variables are used to avoid multicollinearity.

- * Ordinal / Label Encoding:

Assigns integer values to categories based on their natural order. Should be used only when an inherent ranking exists.

- * Count / Frequency Encoding:

Replaces each category with its count or frequency in the dataset. Assumes that category popularity is predictive of the target

Feature Engineering

Feature engineering involves creating new features or transforming existing ones to better capture underlying patterns in the data. Rather than relying solely on raw attributes, engineered features can represent domain knowledge and relationships more effectively. Feature engineering can include combining multiple attributes, deriving indicator variables, or generating new features based on existing data. This process enhances the expressive power of the dataset and can significantly improve model accuracy and generalization.

Data Visualization

Data visualization is an important analytical step used to visually explore and understand the characteristics of a dataset. Visualization techniques help reveal patterns, trends, distributions, and potential anomalies in the data. Univariate visualizations are used to analyze individual feature distributions, while bivariate visualizations help study relationships between pairs of variables. Visualization also aids in validating preprocessing steps such as normalization and encoding, ensuring that transformations have been applied correctly.

The figure below illustrates the pipeline of the experiment, showing the sequence of steps involved in data preprocessing and feature engineering, starting from raw data and resulting in preprocessed data ready for machine learning models.



Code:**## importing libraries**

```
# importing required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing
import MinMaxScaler
```

loading dataset

```
# # Load the Titanic dataset

import pandas as pd

file_path = "Titanic-Dataset.csv"

df = pd.read_csv(file_path)

print("Dataset loaded successfully")

# Display the first 5 rows of the dataset

print("\nFirst 5 rows of the dataset:")

display(df.head())
```

dataset analysis

```
# Display column names along with their data types

df.info()

# Showing the number of rows and columns in the

dataset df.shape
```

Data Preprocessing & Feature Engineering**### Handling Missing Values**

```
# Visualize and display missing values in each column

missing_count = df.isna().sum()

print(" Missing values count per column:")

display(missing_count.to_frame(name="Missing Count"))
```

BAR GRAPH OF MISSING VALUES

```
# Plot the percentage of missing values for each column
missing_percentage = (df.isna().mean() * 100).sort_values(ascending=False)

plt.figure()

missing_percentage.plot(kind="bar")

plt.title("Percentage of Missing Values per Column")

plt.ylabel("Percentage (%)")

plt.xlabel("Columns")

plt.xticks(rotation=45)

plt.show()
```

AGE WITH MEAN

```
# Fill missing Age values using mean
```

```
temp_df = df.copy()

temp_df["Age"] = temp_df["Age"].fillna(temp_df["Age"].mean())

print(" Age filled using Mean")

print("Remaining missing values in Age:", temp_df["Age"].isna().sum())
display(temp_df[["Age"]].head())
```

AGE AFTER FILLING

```
# remaining missing value in age after filling
```

```
df["Age"] = df["Age"].fillna(df["Age"].mean())

print("Remaining missing values in Age:", df["Age"].isna().sum())
```

EMBARKED WITH MOST FREQUENT VALUES

```
# Find the most frequent value in Embarked
```

```
most_frequent_embarked = df["Embarked"].mode()[0]

print("Most frequent Embarked value:",

most_frequent_embarked)
```

```
# Fill missing Embarked values with the most frequent value
```

```
df_embarked = df.copy()
```

```

df_embarked["Embarked"] =
df_embarked["Embarked"].fillna(most_frequent_embarked) print("Remaining missing
values in Embarked:", df_embarked["Embarked"].isna().sum())
display(df_embarked[["Embarked"]].head())
# as there are many null values we Drop the Cabin column df =
df.drop(columns=["Cabin"], errors="ignore") df.head()

```

COLUMNS AFTER MISSING VALUES (HANDLING EACH CASE) TO CHECK IS THERE ANY MISSING VALUE IN DATA OR NOT

```

# Create a cleaned dataset and confirm there are no missing
values df_clean = df.copy()
df_clean["Age"] = df_clean["Age"].fillna(df_clean["Age"].median()) df_clean["Embarked"]
= df_clean["Embarked"].fillna(df_clean["Embarked"].mode()[0]) print("Total missing
values after handling:", df_clean.isna().sum().sum())

```

Normalization

```

# Select numerical columns to apply Min-Max scaling
num_cols = ["Age", "Fare"]

df_num = df_clean[num_cols].copy()

```

MIN_MAX

```

# Scale numerical columns to the 0–1 range using MinMaxScaler
scaler = MinMaxScaler()

df_scaled = df_num.copy()

df_scaled[num_cols] = scaler.fit_transform(df_num[num_cols])

```

BEFORE VS AFTER SCALING

```

# Compare Age distribution before and after Min-Max scaling

plt.figure()

plt.hist(df_num["Age"], bins=30, density=True)

plt.title("Age Distribution (Before Scaling)")

```

```
plt.xlabel("Age")
```

```
plt.ylabel("Density")
```

```
plt.show()
```

```
plt.figure()
```

```
plt.hist(df_scaled["Age"], bins=30, density=True)
```

```
plt.title("Age Distribution (After Min-Max Scaling)")
```

```
plt.xlabel("Scaled Age")
```

```
plt.ylabel("Density")
```

```
plt.show()
```

BEFORE VS AFTER SCALING (FARE)

```
# Compare Fare distribution before and after Min-Max scaling
```

```
plt.figure()
```

```
plt.hist(df_num["Fare"], bins=30)
```

```
plt.title("Fare Distribution (Before Scaling)")
```

```
plt.xlabel("Fare")
```

```
plt.ylabel("Frequency")
```

```
plt.show()
```

```
plt.figure()
```

```
plt.hist(df_scaled["Fare"], bins=30)
```

```
plt.title("Fare Distribution (After Min-Max Scaling)")
```

```
plt.xlabel("Scaled Fare")
```

```
plt.ylabel("Frequency")
```

```
plt.show()
```

Encoding Categorical Variables

```
# List and display categorical columns
```

```
categorical_cols = ["Sex", "Embarked", "Pclass"]
```

```
print("Categorical Columns:")
```

```
print(categorical_cols)
```

binary encoding for Sex

Apply binary encoding to the 'Sex' column

```
df_sex_encoded = df_clean.copy()
```

```
df_sex_encoded["Sex"] = df_sex_encoded["Sex"].map({"male": 0, "female": 1})
```

```
print("Sex column after Binary Encoding:") display(df_sex_encoded[["Sex"]].head())
```

One-Hot encoding for Embarked and Pclass #

One-Hot Encode the 'Embarked' column

```
df_embarked_encoded = df_clean.copy()
```

```
df_embarked_encoded = pd.get_dummies(df_embarked_encoded, columns=["Embarked"],  
prefix="Embarked")
```

```
print("Embarked column after One-Hot Encoding:")
```

```
display(df_embarked_encoded.filter(like="Embarked").head())
```

One-Hot Encode the 'Pclass' column

```
df_pclass_encoded = df_clean.copy()
```

```
df_pclass_encoded = pd.get_dummies(df_pclass_encoded, columns=["Pclass"], prefix="Pclass")
```

```
print("Pclass column after One-Hot Encoding:") display(df_pclass_encoded.filter(like="Pclass").head())
```

Visualization

Univariate Visualization

- **Survival count plot**

Plot the count of passengers who survived vs. did not survive

```
plt.figure()
```

```
df_clean["Survived"].value_counts().sort_index().plot(kind="bar")
```

```
plt.title("Survival Count")
```

```
plt.xlabel("Survived (0 = No, 1 = Yes)")
```

```
plt.ylabel("Count")
```

```
plt.show()
```


Age distribution (histogram)

Plot the distribution of passenger ages

```
plt.figure()
plt.hist(df_clean["Age"], bins=30)
plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```

Bivariate Visualization

Plot a grouped bar chart of survival counts by sex

```
plt.figure()
df_clean.groupby(["Sex", "Survived"]).size().unstack().plot(kind="bar")
plt.title("Survival vs Sex")
plt.xlabel("Sex")
plt.ylabel("Count")
plt.show()
```

Plot a grouped bar chart of survival counts by passenger class

```
plt.figure()
df_clean.groupby(["Pclass", "Survived"]).size().unstack().plot(kind="bar")
plt.title("Survival vs Pclass")
plt.xlabel("Passenger Class")
plt.ylabel("Count")
plt.show()
```

Compare Fare distribution by survival status using a box plot

```
plt.figure()
df_clean.boxplot(column="Fare", by="Survived")
plt.title("Fare vs Survival")
plt.suptitle("") # Remove automatic subtitle
plt.xlabel("Survived (0 = No, 1 = Yes)")
```

```
plt.ylabel("Fare")
```

```
plt.show()
```

Feature Engineering (New Features)

```
new features like FamilySize (FamilySize = SibSp + Parch + 1) #
```

```
Create a new feature 'FamilySize' as SibSp + Parch + 1
```

```
df_family = df_clean.copy()
```

```
df_family["FamilySize"] = df_family["SibSp"] + df_family["Parch"] + 1
```

```
print("FamilySize feature created:") display(df_family[["SibSp",  
"Parch", "FamilySize"]].head())
```

```
new features like IsAlone = 1 if FamilySize == 1 else 0
```

```
# Create a new feature 'IsAlone' where 1 indicates traveling alone
```

```
df_isalone = df_family.copy()
```

```
df_isalone["IsAlone"] = (df_isalone["FamilySize"] == 1).astype(int)
```

```
print("IsAlone feature created:")
```

```
display(df_isalone[["FamilySize", "IsAlone"]].head()) #
```

```
Add 'FamilySize' and 'IsAlone' features to the dataset
```

```
df_clean["FamilySize"] = df_clean["SibSp"] + df_clean["Parch"] + 1 df_clean["IsAlone"]
```

```
= (df_clean["FamilySize"] == 1).astype(int)
```

```
display(df_clean[["FamilySize", "IsAlone"]].head())
```

Final dataset

```
# final cleaned dataset
```

```
df.describe()
```

RESULT:

Display column names along with their data types

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 891 entries, 0 to 890
```

```
Data columns (total 12 columns):
```

#	Column	Non-Null Count	Dtype
0	PassengerId	891 non-null	int64
1	Survived	891 non-null	int64
2	Pclass	891 non-null	int64
3	Name	891 non-null	object
4	Sex	891 non-null	object
5	Age	714 non-null	float64
6	SibSp	891 non-null	int64
7	Parch	891 non-null	int64
8	Ticket	891 non-null	object
9	Fare	891 non-null	float64
10	Cabin	204 non-null	object
11	Embarked	889 non-null	object

```
dtypes: float64(2), int64(5), object(5)
```

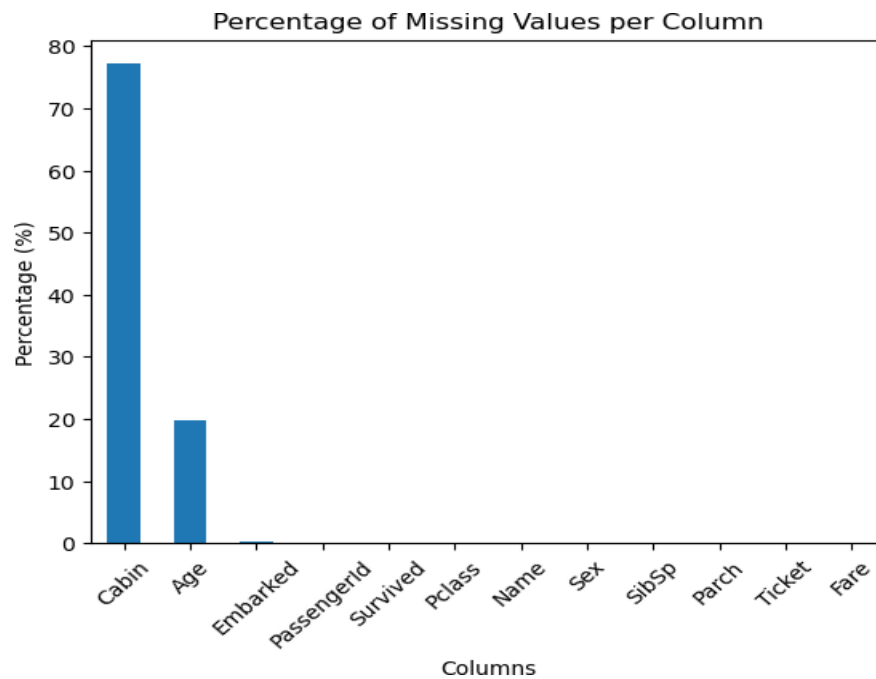
```
memory usage: 83.7+ KB
```

Missing values count per column:

Missing values count per column:

Missing Count	
PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	177
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	687
Embarked	2

BAR GRAPH OF MISSING VALUES



Age filled using Mean

Remaining missing values in Age: 0

	Age
0	22.0
1	38.0
2	26.0
3	35.0
4	35.0

Most frequent Embarked value: S

Remaining missing values in Embarked: 0

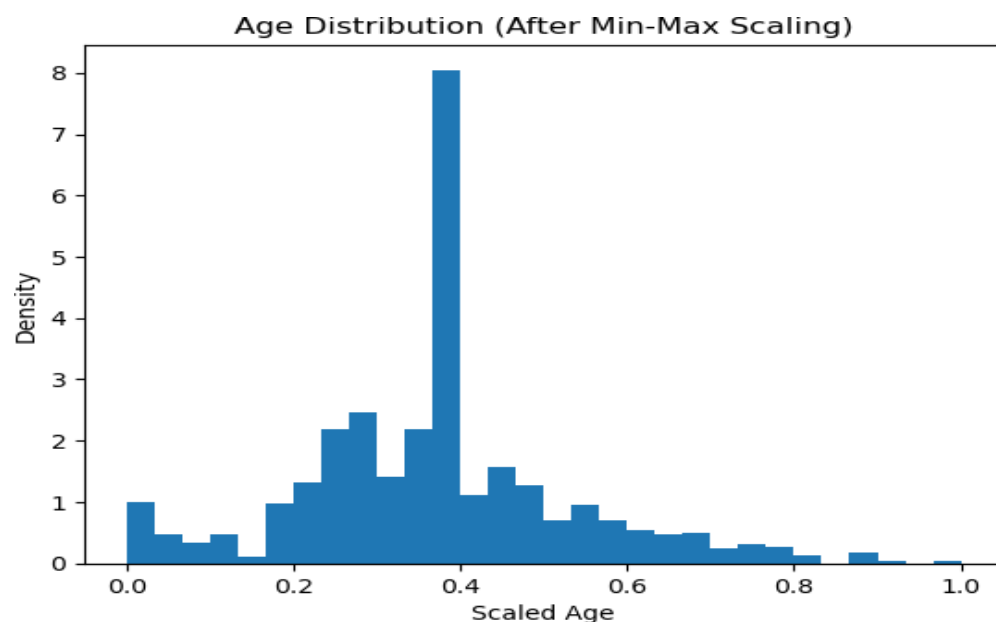
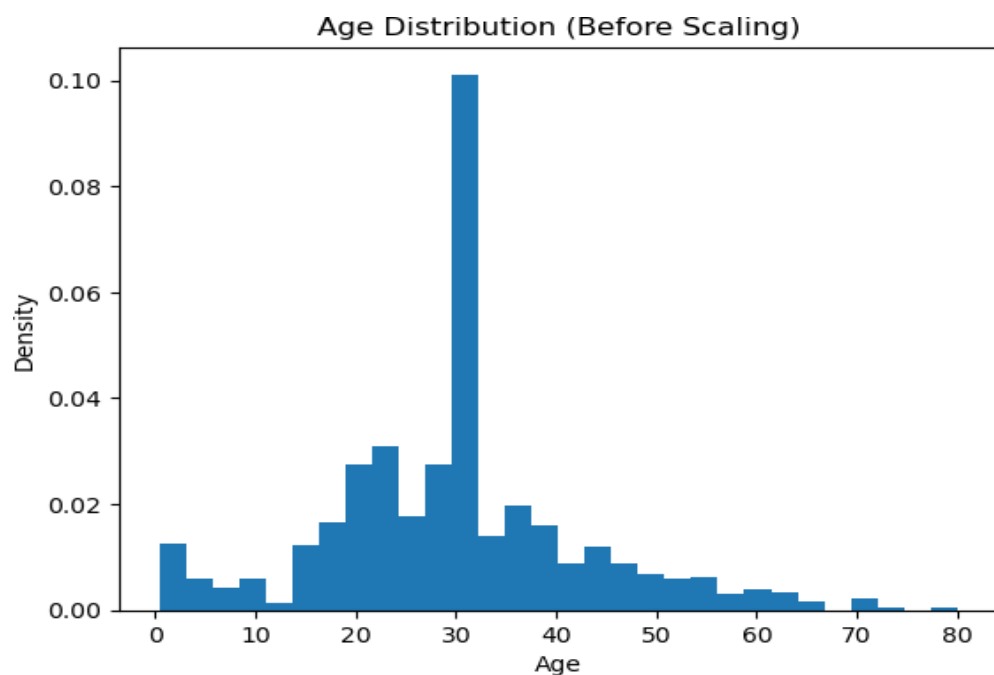
Embarked	
0	S
1	C
2	S
3	S
4	S

#as there are many null values we Drop
the Cabin column

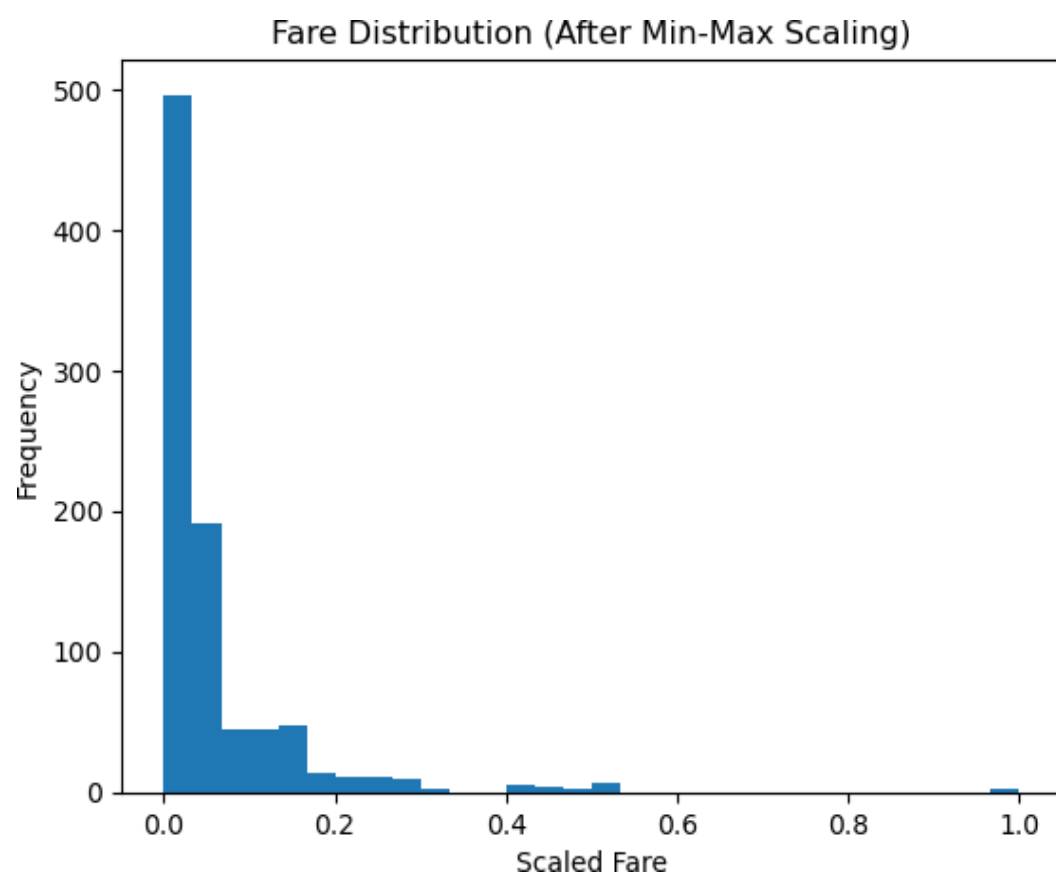
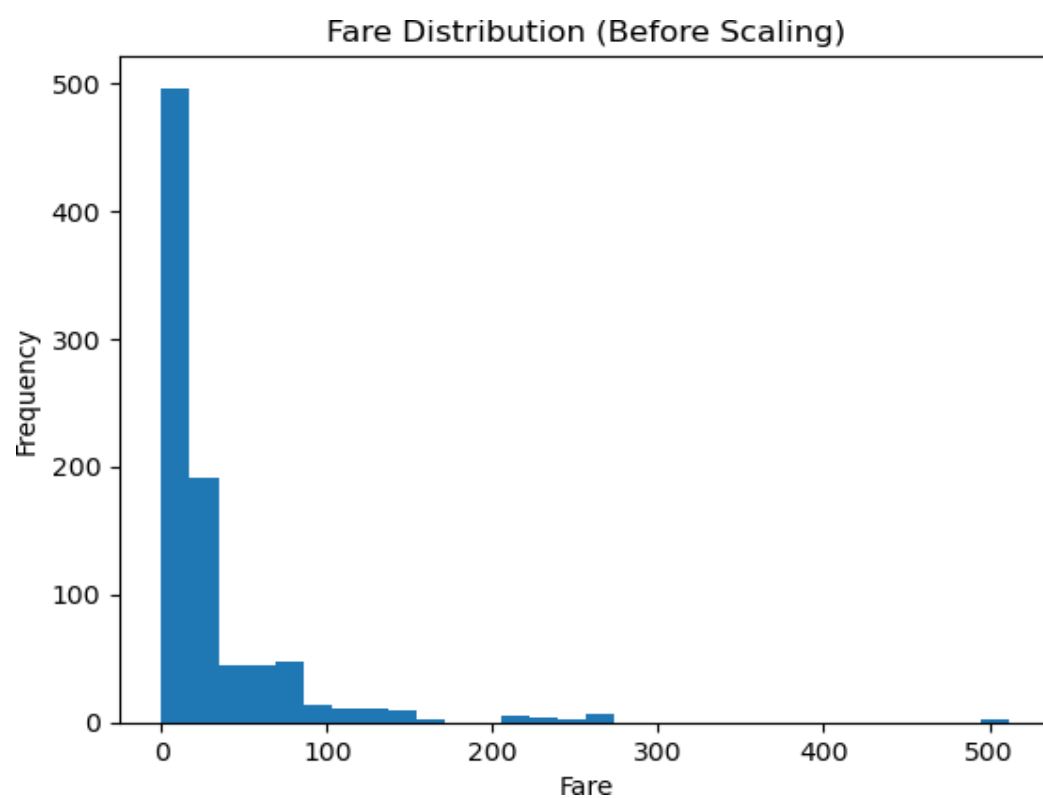
	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	S

Normalization

BEFORE VS AFTER SCALING



BEFORE VS AFTER SCALING (FARE)



Encoding Categorical Variables

Categorical Columns:

['Sex', 'Embarked', 'Pclass']

Sex column after Binary Encoding:

	Sex
0	0
1	1
2	1
3	1
4	0

Embarked column after One-Hot Encoding:

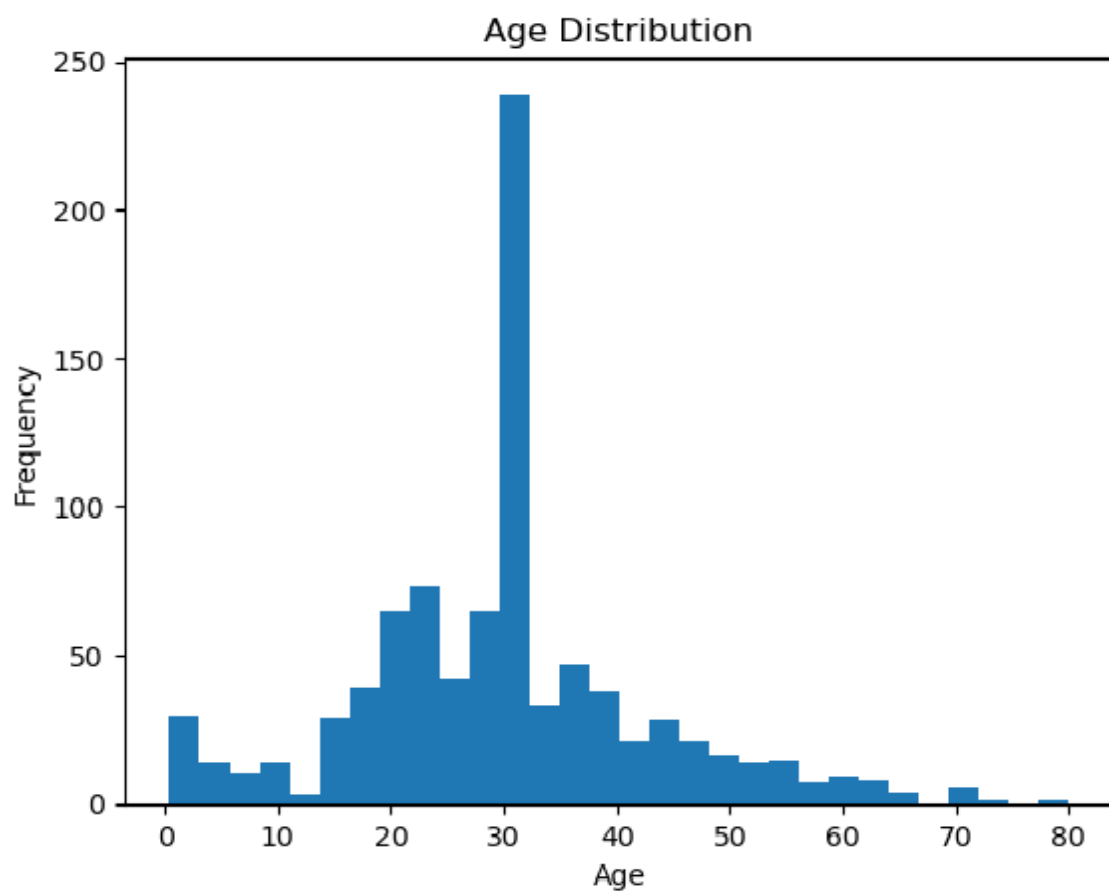
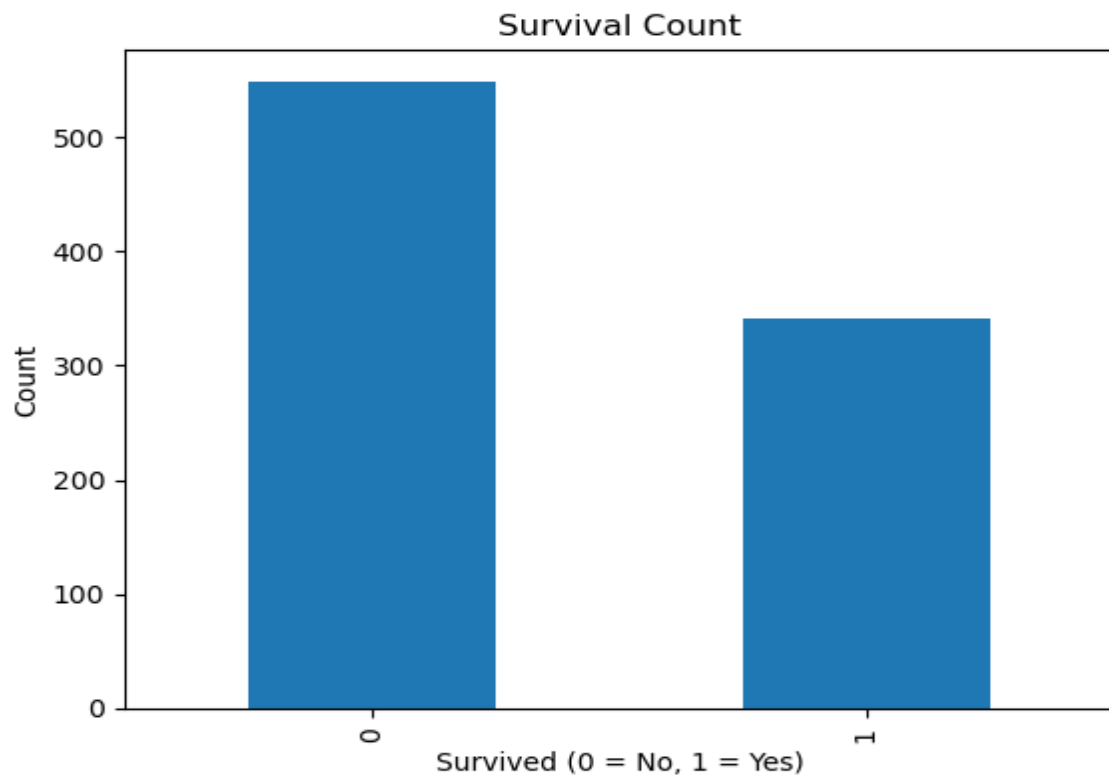
	Embarked_C	Embarked_Q	Embarked_S
0	False	False	True
1	True	False	False
2	False	False	True
3	False	False	True
4	False	False	True

Pclass column after One-Hot Encoding:

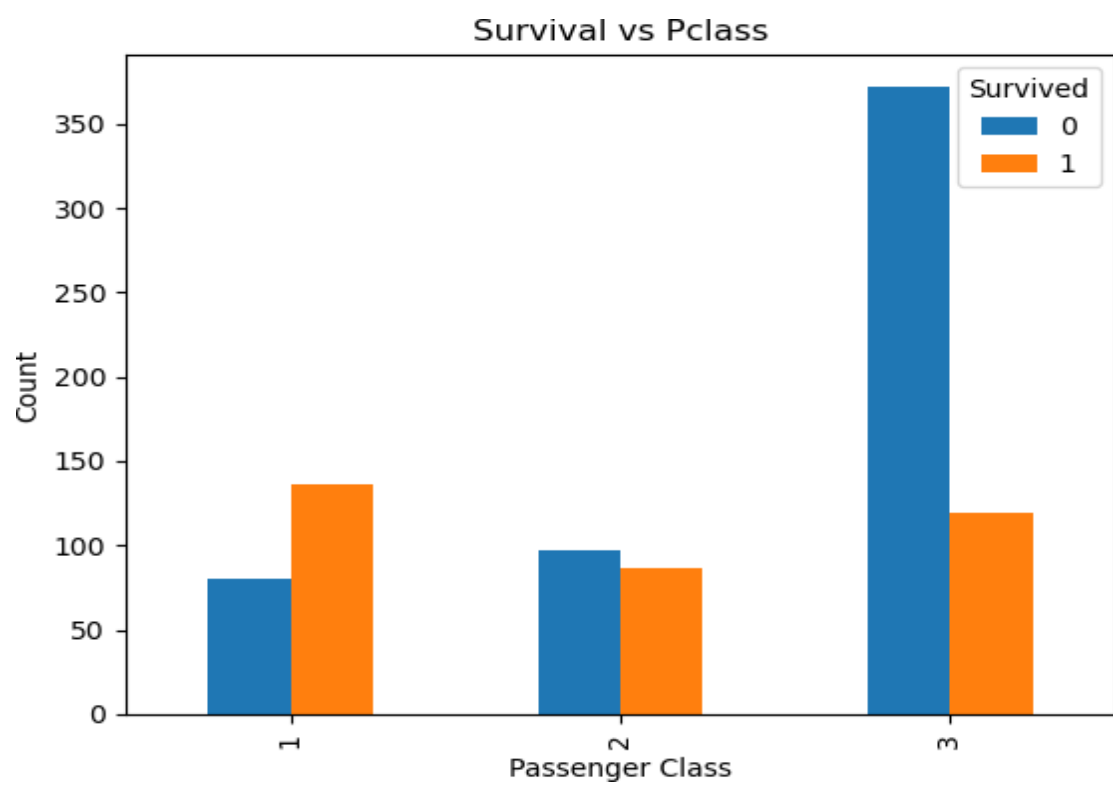
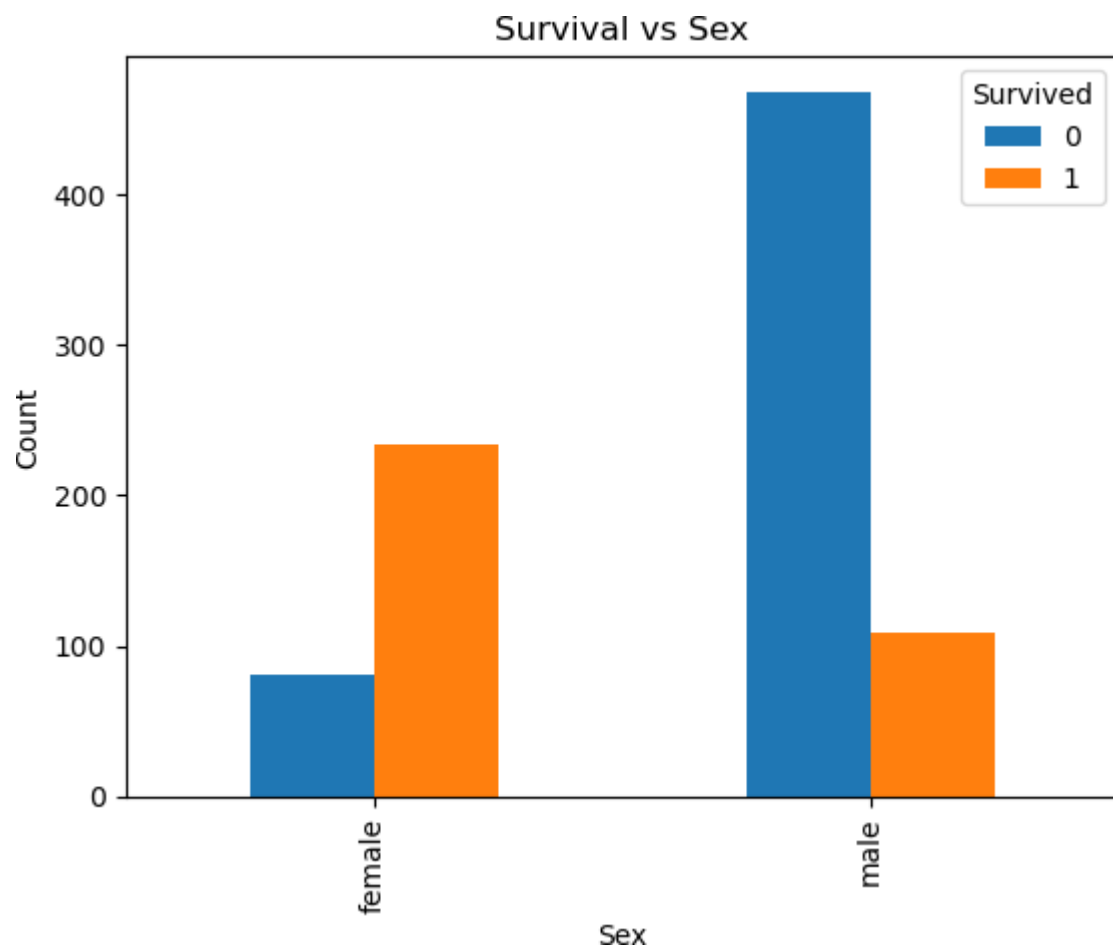
	Pclass_1	Pclass_2	Pclass_3
0	False	False	True
1	True	False	False
2	False	False	True
3	True	False	False
4	False	False	True

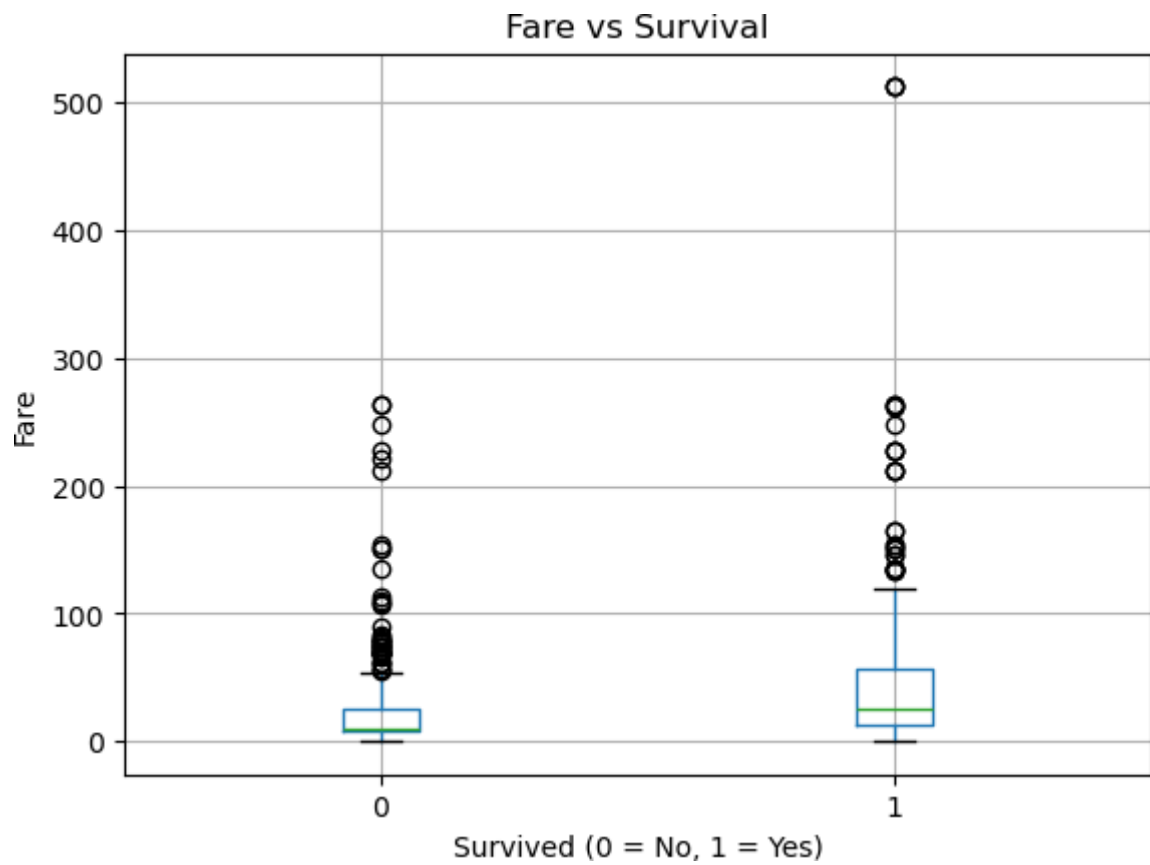
Visualization

Univariate Visualization



Bivariate Visualization





FamilySize feature created:

	SibSp	Parch	FamilySize
0	1	0	2
1	1	0	2
2	0	0	1
3	1	0	2
4	0	0	1

IsAlone feature created:

IsAlone	
0	0
1	0
2	1
3	0
4	1

FamilySize	IsAlone
0	2
1	2
2	1
3	2
4	1

Final cleaned dataset

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	891.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	13.002015	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	22.000000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	29.699118	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	35.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

Data Preprocessing - Practical Implementation

Virtual Labs - Dayalbagh Educational Institute

Step 1: Importing Libraries

Importing Required Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import MinMaxScaler
print("Libraries Imported")
```

Output:

Libraries Imported

Step 2: Loading Dataset

Load the Titanic dataset

```
file_path = "Titanic-Dataset.csv"
data = pd.read_csv(file_path)
print("Dataset loaded successfully")
```

Output:

Dataset loaded successfully

Step 3: Data Analysis

Display the first 5 rows of the dataset

```
data.head()
```

Output:

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
0 1	0	3	Braund, Mr. Owen H	male	22.0	1	0	A/5 21171
7.2500	NaN	S						
1 2	1	1	Cumings, Mrs. John	female	38.0	1	0	PC 17599
71.2833	C85	C						
2 3	1	3	Heikkinen, Miss. L	female	26.0	0	0	STON/O2.
3101282	7.9250	NaN	S					
3 4	1	1	Futrelle, Mrs. Jac	female	35.0	1	0	113803
53.1000	C123	S						
4 5	0	3	Allen, Mr. William	male	35.0	0	0	373450
8.0500	NaN	S						

Display dataset structure and data types

```
data.info()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype
---  -
0   PassengerId     891 non-null   int64
1   Survived        891 non-null   int64
2   Pclass         891 non-null   int64
3   Name            891 non-null   object
4   Sex             891 non-null   object
5   Age            714 non-null   float64
6   SibSp          891 non-null   int64
7   Parch          891 non-null   int64
8   Ticket         891 non-null   object
9   Fare           891 non-null   float64
10  Cabin          204 non-null   object
11  Embarked       889 non-null   object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

Show the number of rows and columns in the dataset

```
data.shape
```

Output:

```
(891, 12)
```

Count occurrences of each class in the Survived column

```
data['Survived'].value_counts()
```

Output:

```
0    5491    342Name: Survived, dtype: int64
```

Count frequency of each Sex value

```
data['Sex'].value_counts()
```

Output:

```
Sex
male    577
female  314
Name: Sex, dtype: int64
```

Step 4: Data Preprocessing

(HANDLING MISSING VALUES) Visualize and display missing values in each column

```
missing_count = data.isna().sum()
print("Missing values count per column:")
display(missing_count.to_frame(name="Missing Count"))
```

Output:

Missing values count per column:

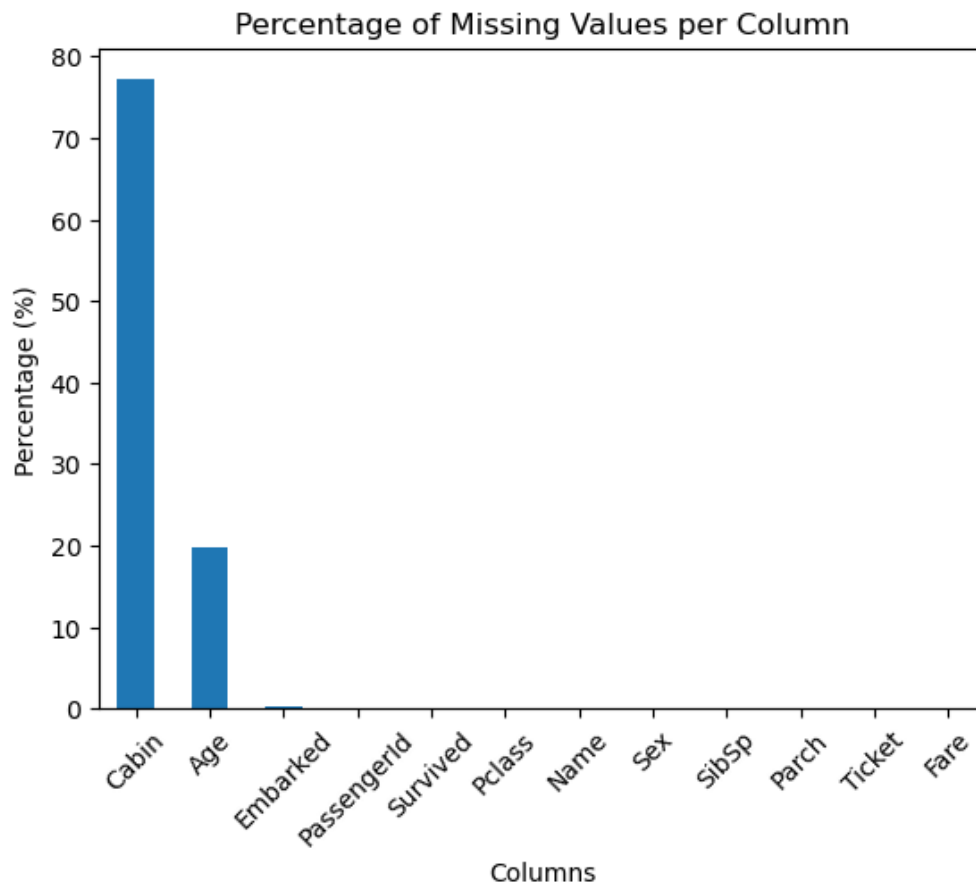
Missing Count

PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	177
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	687
Embarked	2

Plot the percentage of missing values for each column

```
missing_percentage = (df.isna().mean() * 100).sort_values(ascending=False)

plt.figure()
missing_percentage.plot(kind="bar")
plt.title("Percentage of Missing Values per Column")
plt.ylabel("Percentage (%)")
plt.xlabel("Columns")
plt.xticks(rotation=45)
plt.show()
```



Fill missing Age values using mean

```
temp_df = df.copy()
temp_df["Age"] = temp_df["Age"].fillna(temp_df["Age"].mean())

print("Age filled using Mean")
display(temp_df[["Age"]].head())
```

Output:

```
Age filled using Mean
Age
0    22.0
1    38.0
2    26.0
3    35.0
4    35.0
```

Remaining missing value in age after filling

```
df["Age"] = df["Age"].fillna(df["Age"].mean())
print("Remaining missing values in Age:", df["Age"].isna().sum())
```

Output:

```
Remaining missing values in Age: 0
```

Find the most frequent value in Embarked

```
most_frequent_embarked = df["Embarked"].mode()[0]
print("Most frequent Embarked value:", most_frequent_embarked)
```


Output:

Most frequent Embarked value: S

Fill missing Embarked values with the most frequent value

```
df_embarked = df.copy()
df_embarked["Embarked"] = df_embarked["Embarked"].fillna(most_frequent_embarked)

print("Remaining missing values in Embarked:", df_embarked["Embarked"].isna().sum())
display(df_embarked[["Embarked"]].head())
```

Output:

Remaining missing values in Embarked: 0
 Embarked
 0 S
 1 C
 2 S
 3 S
 4 S

as there are many null values we Drop the Cabin column

```
df = df.drop(columns=["Cabin"], errors="ignore")
df.head()
```

Output:

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
0 1	0	3	Braund, Mr. Owen H	male	22.0	1	0	A/5 21171
7.2500	S							
1 2	1	1	Cumings, Mrs. John	female	38.0	1	0	PC 17599
71.2833	C							
2 3	1	3	Heikkinen, Miss. L	female	26.0	0	0	STON/O2.
3101282	7.9250	S						
3 4	1	1	Futrelle, Mrs. Jac	female	35.0	1	0	113803
53.1000	S							
4 5	0	3	Allen, Mr. William	male	35.0	0	0	373450
8.0500	S							

Create a cleaned dataset and confirm there are no missing values

```
df_clean = df.copy()
df_clean["Age"] = df_clean["Age"].fillna(df_clean["Age"].median())
df_clean["Embarked"] = df_clean["Embarked"].fillna(df_clean["Embarked"].mode()[0])
print("Total missing values after handling:", df_clean.isna().sum().sum())
```

Output:

Total missing values after handling: 0

(NORMALIZATION) Select numerical columns to apply Min-Max scaling

```
num_cols = ["Age", "Fare"]
df_num = df_clean[num_cols].copy()
print("MinMaxScaler imported and numerical columns selected")
```

Output:

MinMaxScaler imported and numerical columns selected

Scale numerical columns to the 0-1 range using MinMaxScaler

```

scaler = MinMaxScaler()

df_scaled = df_num.copy()
df_scaled[num_cols] = scaler.fit_transform(df_num[num_cols])
print("Numerical columns balanced and scaled between 0 and 1")

```

Output:

Numerical columns balanced and scaled between 0 and 1

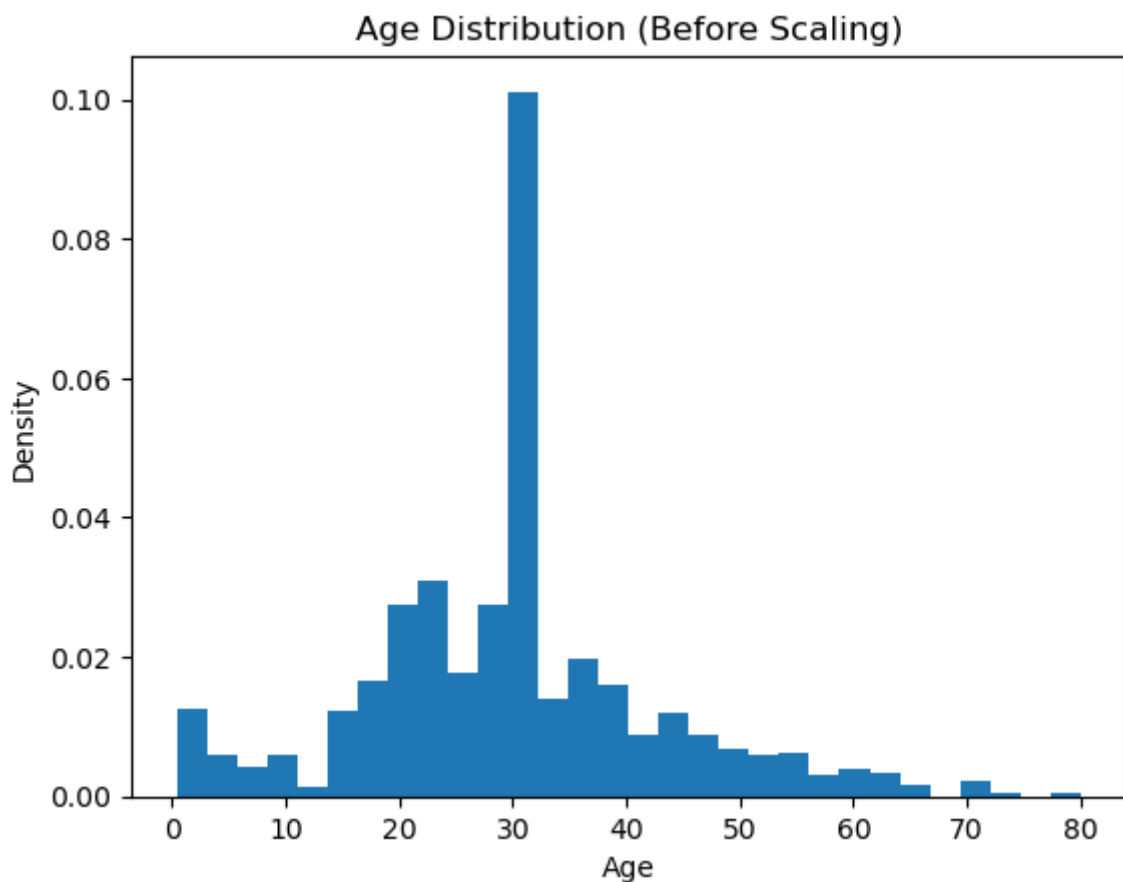
Compare Age distribution before and after Min-Max scaling

```

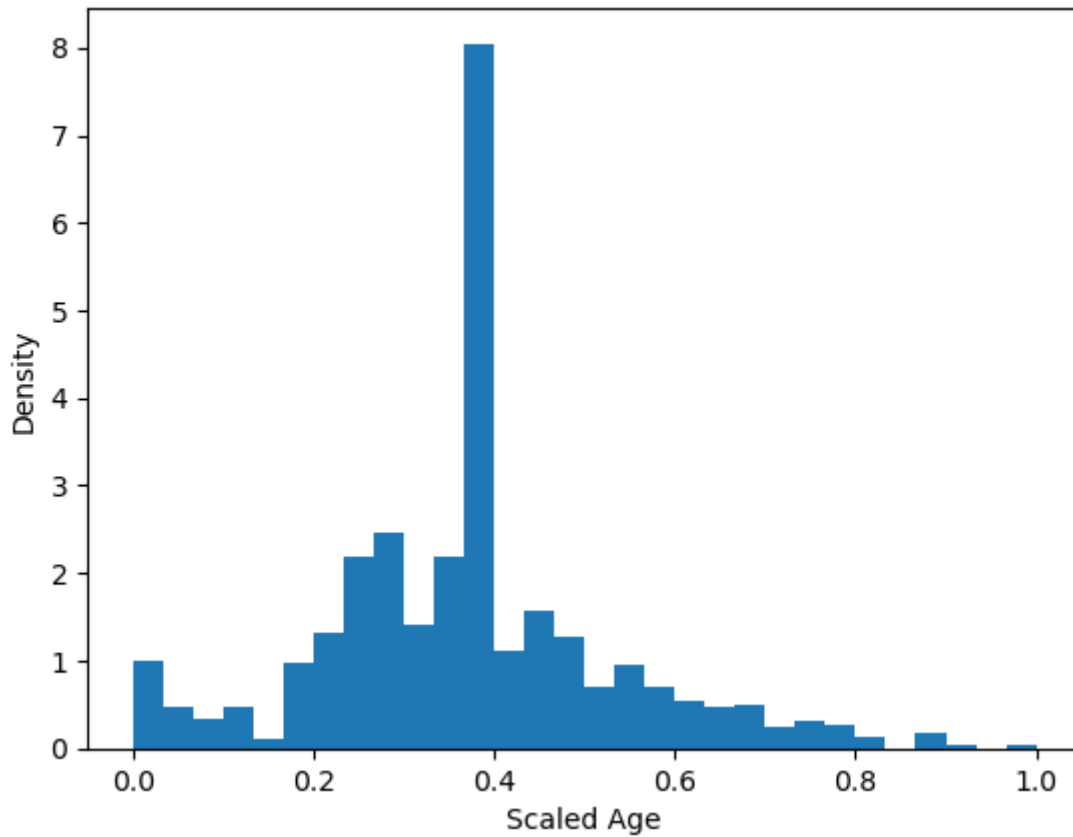
plt.figure()
plt.hist(df_num["Age"], bins=30, density=True)
plt.title("Age Distribution (Before Scaling)")
plt.xlabel("Age")
plt.ylabel("Density")
plt.show()

plt.figure()
plt.hist(df_scaled["Age"], bins=30, density=True)
plt.title("Age Distribution (After Min-Max Scaling)")
plt.xlabel("Scaled Age")
plt.ylabel("Density")
plt.show()

```



Age Distribution (After Min-Max Scaling)

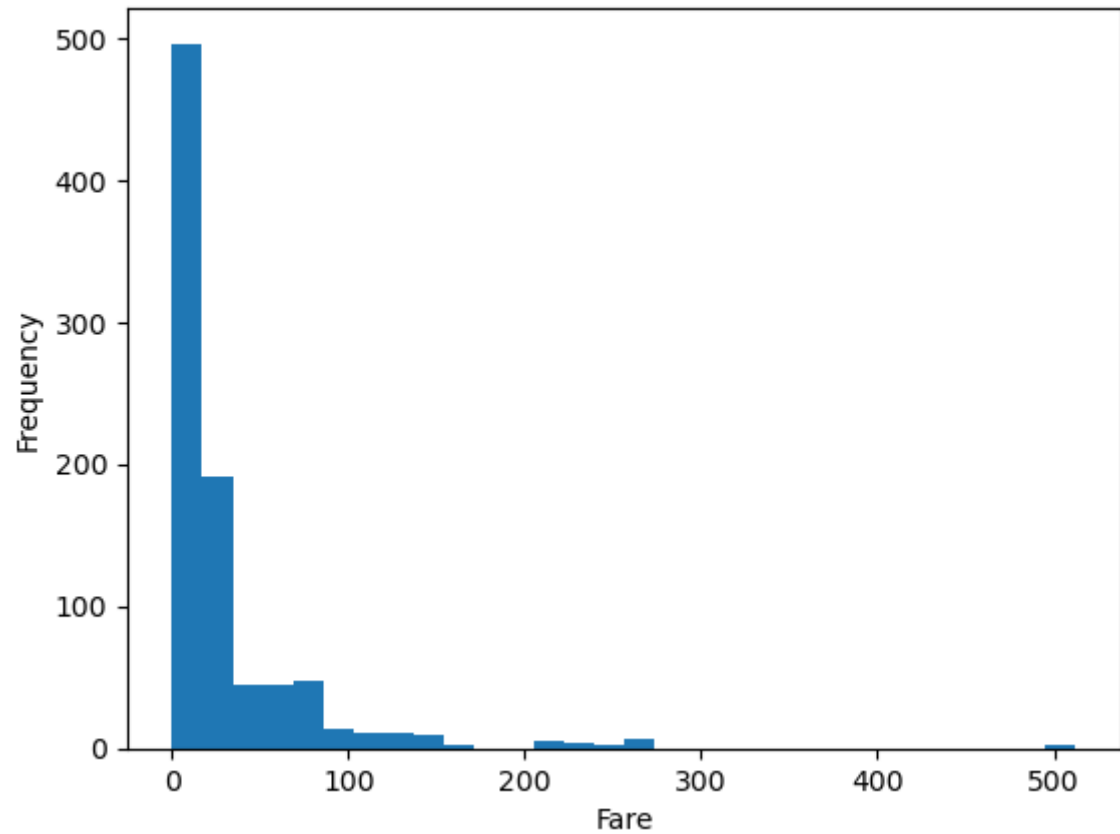


Compare Fare distribution before and after Min-Max scaling

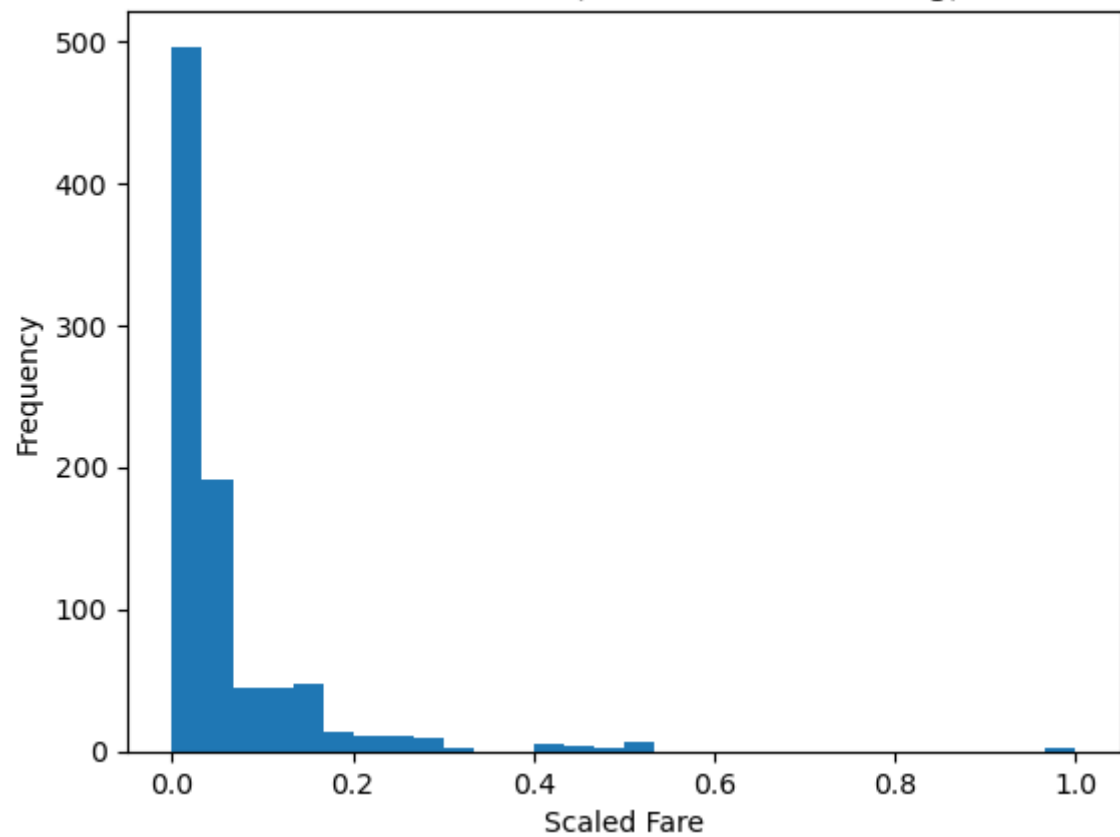
```
plt.figure()
plt.hist(df_num["Fare"], bins=30)
plt.title("Fare Distribution (Before Scaling)")
plt.xlabel("Fare")
plt.ylabel("Frequency")
plt.show()

plt.figure()
plt.hist(df_scaled["Fare"], bins=30)
plt.title("Fare Distribution (After Min-Max Scaling)")
plt.xlabel("Scaled Fare")
plt.ylabel("Frequency")
plt.show()
```

Fare Distribution (Before Scaling)



Fare Distribution (After Min-Max Scaling)



```
# (ENCODING CATEGORICAL VARIABLES) List and display categorical columns
```

```
categorical_cols = ["Sex", "Embarked", "Pclass"]

print("Categorical Columns:")
print(categorical_cols)
```

Output:

```
Categorical Columns:
['Sex', 'Embarked', 'Pclass']
```

Apply binary encoding to the 'Sex' column

```
df_sex_encoded = df_clean.copy()
df_sex_encoded["Sex"] = df_sex_encoded["Sex"].map({"male": 0, "female": 1})

print("Sex column after Binary Encoding:")
display(df_sex_encoded[["Sex"]].head())
```

Output:

```
Sex column after Binary Encoding:
Sex
0  0
1  1
2  1
3  1
4  0
```

One-Hot Encode the 'Embarked' column

```
df_embarked_encoded = df_clean.copy()
df_embarked_encoded = pd.get_dummies(df_embarked_encoded, columns=["Embarked"],
    prefix="Embarked")

print("Embarked column after One-Hot Encoding:")
display(df_embarked_encoded.filter(like="Embarked").head())
```

Output:

```
Embarked column after One-Hot Encoding:
Embarked_C  Embarked_Q  Embarked_S
0  False          False         True
1   True          False         False
2  False          False         True
3  False          False         True
4  False          False         True
```

One-Hot Encode the 'Pclass' column

```
df_pclass_encoded = df_clean.copy()
df_pclass_encoded = pd.get_dummies(df_pclass_encoded, columns=["Pclass"], prefix="Pclass")

print("Pclass column after One-Hot Encoding:")
display(df_pclass_encoded.filter(like="Pclass").head())
```

Output:

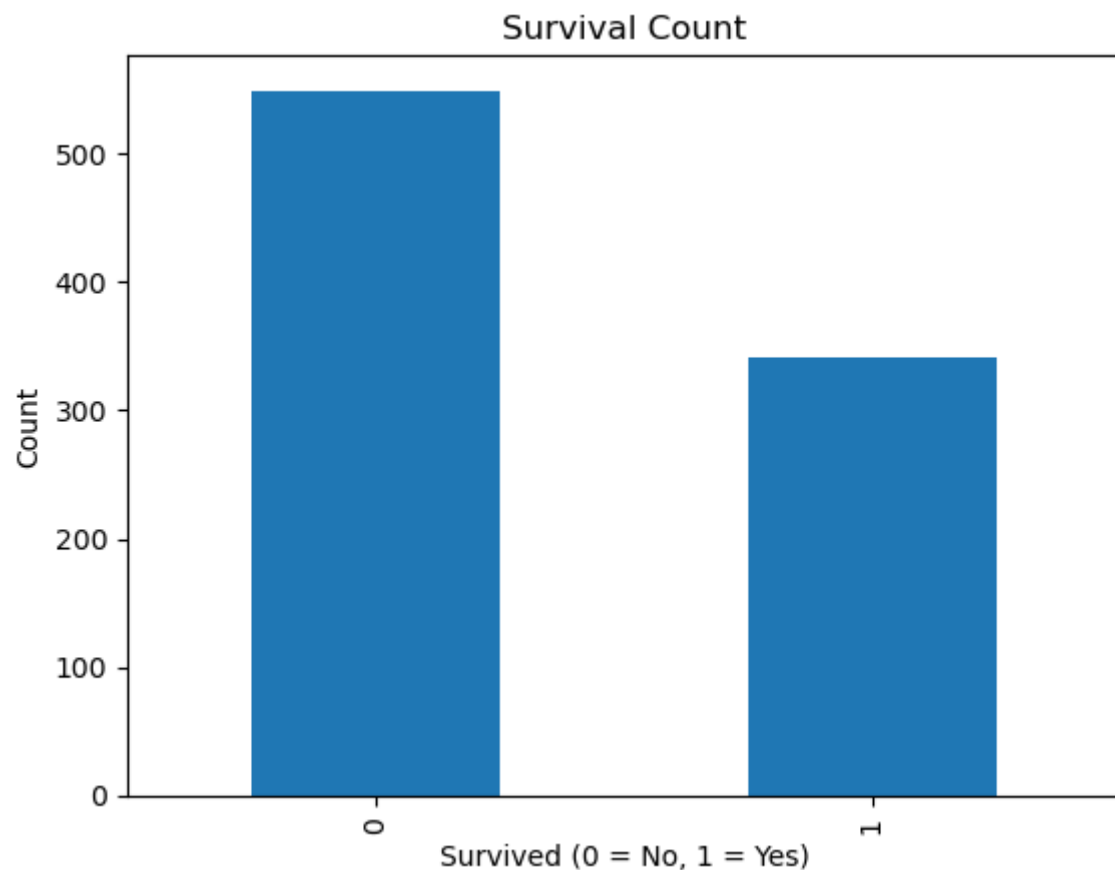
Pclass column after One-Hot Encoding:

	Pclass_1	Pclass_2	Pclass_3
0	False	False	True
1	True	False	False
2	False	False	True
3	True	False	False
4	False	False	True

Step 5: Visualization

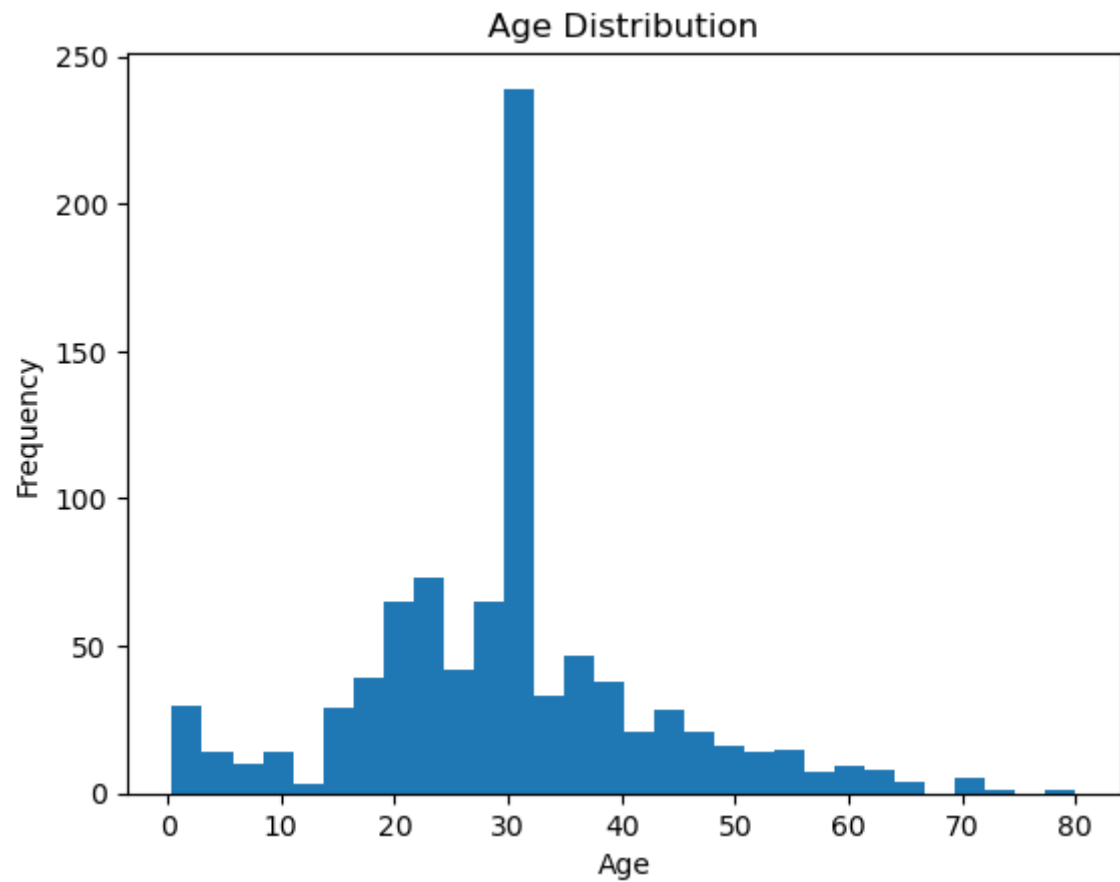
(UNIVARIATE VISUALIZATION) Plot the count of passengers who survived vs. did not survive

```
plt.figure()
df_clean["Survived"].value_counts().sort_index().plot(kind="bar")
plt.title("Survival Count")
plt.xlabel("Survived (0 = No, 1 = Yes)")
plt.ylabel("Count")
plt.show()
```



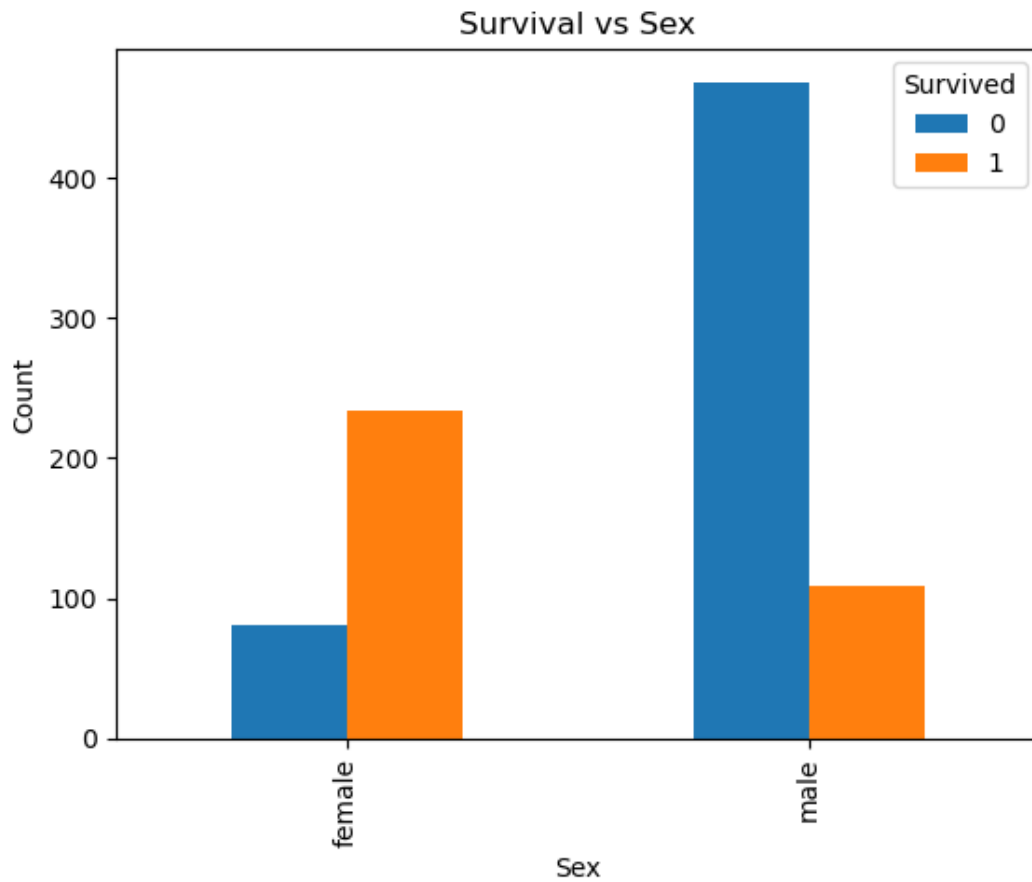
Plot the distribution of passenger ages

```
plt.figure()
plt.hist(df_clean["Age"], bins=30)
plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```



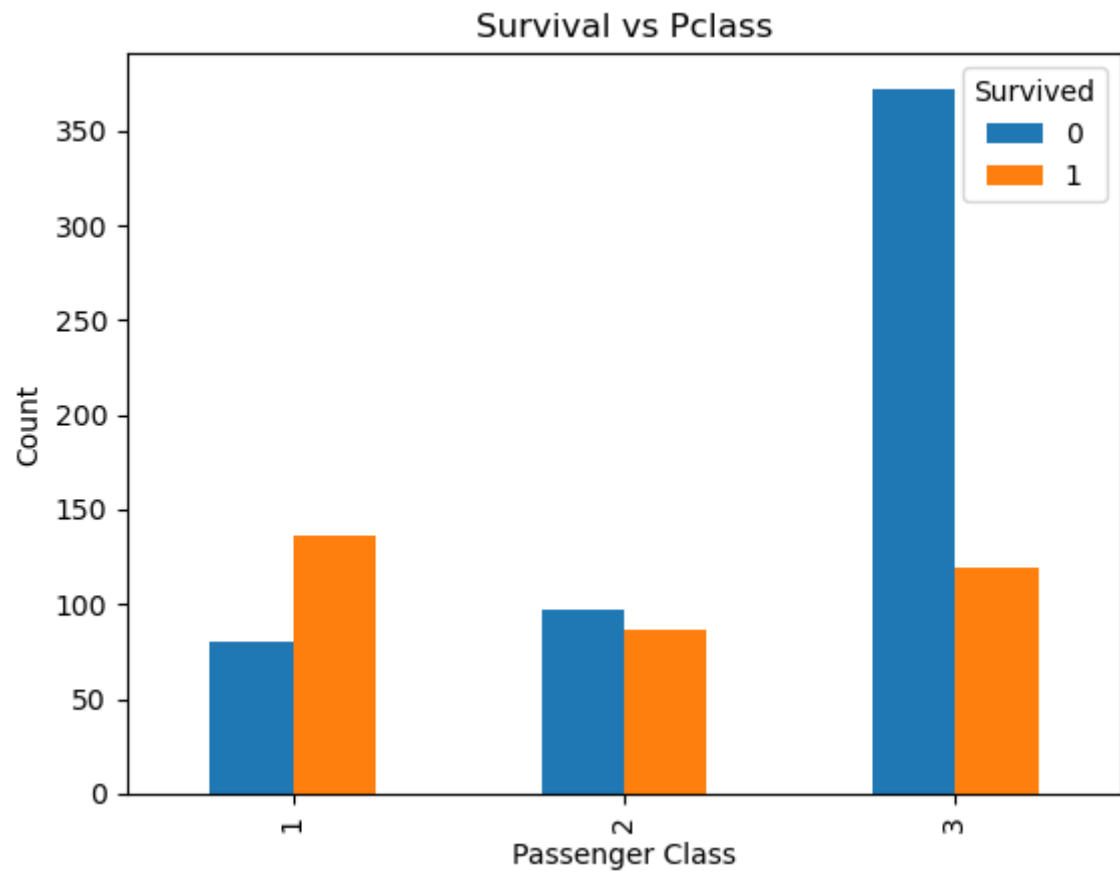
(BIVARIATE VISUALIZATION) Plot a grouped bar chart of survival counts by sex

```
plt.figure()
df_clean.groupby(["Sex", "Survived"]).size().unstack().plot(kind="bar")
plt.title("Survival vs Sex")
plt.xlabel("Sex")
plt.ylabel("Count")
plt.show()
```



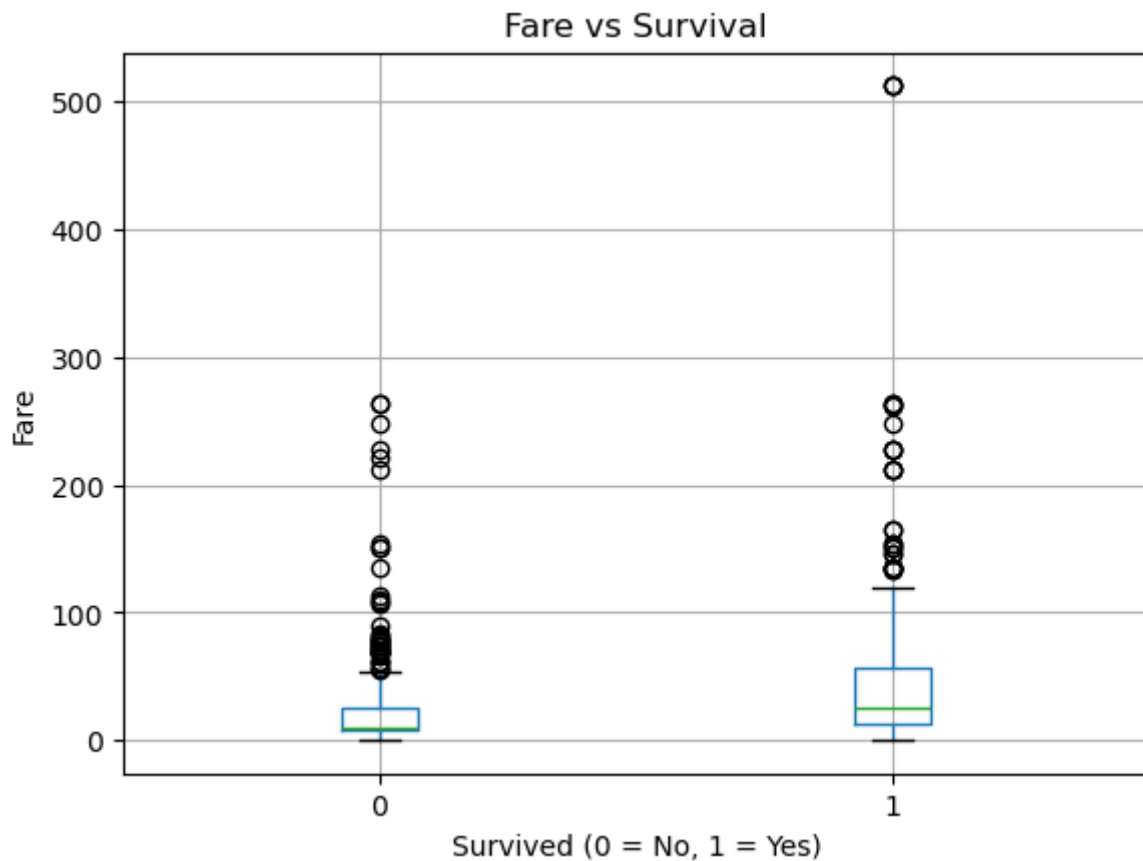
Plot a grouped bar chart of survival counts by passenger class

```
plt.figure()
df_clean.groupby(["Pclass", "Survived"]).size().unstack().plot(kind="bar")
plt.title("Survival vs Pclass")
plt.xlabel("Passenger Class")
plt.ylabel("Count")
plt.show()
```

Compare Fare distribution by survival status using a box plot

```
plt.figure()
df_clean.boxplot(column="Fare", by="Survived")
plt.title("Fare vs Survival")
plt.suptitle("")
plt.xlabel("Survived (0 = No, 1 = Yes)")
plt.ylabel("Fare")
plt.show()
```



Step 6: Feature Engineering

Create a new feature 'FamilySize' as SibSp + Parch + 1

```
df_family = df_clean.copy()
df_family["FamilySize"] = df_family["SibSp"] + df_family["Parch"] + 1

print("FamilySize feature created:")
display(df_family[["SibSp", "Parch", "FamilySize"]].head())
```

Output:

FamilySize feature created:

SibSp	Parch	FamilySize	
0	1	0	2
1	1	0	2
2	0	0	1
3	1	0	2
4	0	0	1

Create a new feature 'IsAlone' where 1 indicates traveling alone

```
df_isalone = df_family.copy()
df_isalone["IsAlone"] = (df_isalone["FamilySize"] == 1).astype(int)

print("IsAlone feature created:")
display(df_isalone[["FamilySize", "IsAlone"]].head())
```

Output:

IsAlone feature created:

	FamilySize	IsAlone
0	2	0
1	2	0
2	1	1
3	2	0
4	1	1

Add 'FamilySize' and 'IsAlone' features to the dataset

```
df_clean["FamilySize"] = df_clean["SibSp"] + df_clean["Parch"] + 1
df_clean["IsAlone"] = (df_clean["FamilySize"] == 1).astype(int)

display(df_clean[["FamilySize", "IsAlone"]].head())
```

Output:

	FamilySize	IsAlone
0	2	0
1	2	0
2	1	1
3	2	0
4	1	1

Final cleaned dataset

```
df_clean.describe()
```

Output:

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	891.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	13.002015	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	22.000000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	29.699118	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	35.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200